TREES

Dr. Priyanka N

Assistant Professor Senior Grade 1

School of Computer Science & Engineering

VIT, Vellore.

Module:1	Algorithm Analysis	8 hours
Importance of algorithms and data structures - Fundamentals of algorithm analysis: Space and time complexity of an algorithm, Types of asymptotic notations and orders of growth - Algorithm efficiency – best case, worst case, average case - Analysis of non-recursive and recursive algorithms - Asymptotic analysis for recurrence relation: Iteration Method, Substitution Method, Master Method and Recursive Tree Method.		
Module:2	**Linear Data Structures**	**7 hours**
Arrays: 1D and 2D array- Stack - Applications of stack: Expression Evaluation, Conversion of Infix to postfix and prefix expression, Tower of Hanoi – Queue - Types of Queue: Circular Queue, Double Ended Queue (deQueue) - Applications – List: Singly linked lists, Doubly linked lists, Circular linked lists- Applications: Polynomial Manipulation.		
Module:3	**Searching and Sorting**	**7 hours**
Searching: Linear Search and binary search – Applications. Sorting: Insertion sort, Selection sort, Bubble sort, Counting sort, Quick sort, Merge sort - Analysis of sorting algorithms.		
Module:4	**Trees**	**6 hours**
Introduction - Binary Tree: Definition and Properties - Tree Traversals- Expression Trees:- Binary Search Trees - Operations in BST: insertion, deletion, finding min and max, finding the k^{th} minimum element.		
Module:5	**Graphs**	**6 hours**
Terminology – Representation of Graph – Graph Traversal: Breadth First Search (BFS), Depth First Search (DFS) - Minimum Spanning Tree: Prim's, Kruskal's - Single Source Shortest Path: Dijkstra's Algorithm.		
Module:6	**Hashing**	**4 hours**
Hash functions - Separate chaining - Open hashing: Linear probing, Quadratic probing, Double hashing - Closed hashing - Random probing – Rehashing - Extendible hashing.		
Module:7	**Heaps and AVL Trees**	**5 hours**
Heaps - Heap sort- Applications -Priority Queue using Heaps. AVL trees: Terminology, basic operations (rotation, insertion and deletion).		

TREES

- Introduction to Trees & Terminologies
- Binary Tree – Definition and Properties
- Binary Tree Representation
- Tree Traversals
- Expression Trees: Binary Search Trees
- Operations on Binary Search Trees
  - Insertion
  - Deletion
  - Finding Max and Minimum
  - Finding the k^{th} Minimum Element

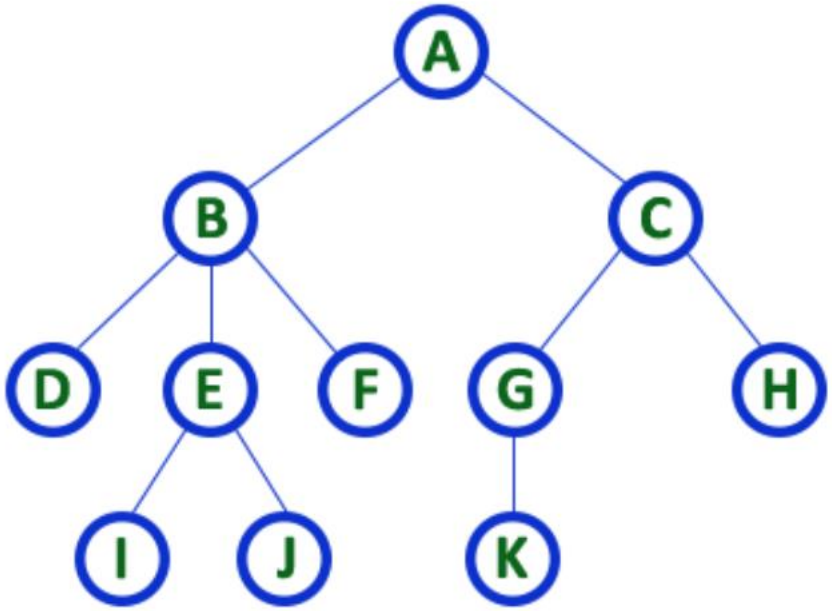
Trees

- Linear data structure - data is organized in sequential order.
- Non-linear data structure - data is organized in random order
- A tree is a very popular non-linear data structure used in a wide range of applications.
- Tree is a non-linear data structure which organizes data in hierarchical structure
- Tree data structure is a collection of data (Node) which is organized in hierarchical structure recursively.

Tree Data Structure

- Every individual element is called as **Node**
- Node in a tree data structure stores the actual data of that particular element and link to next element in hierarchical structure.
- **N** number of nodes then we can have a maximum of **N-1** number of edges.

Tree - Example



- *Tree with 11 nodes and 10 Edges*
- *In any tree with n nodes, there will be maximum of n-1 edges*

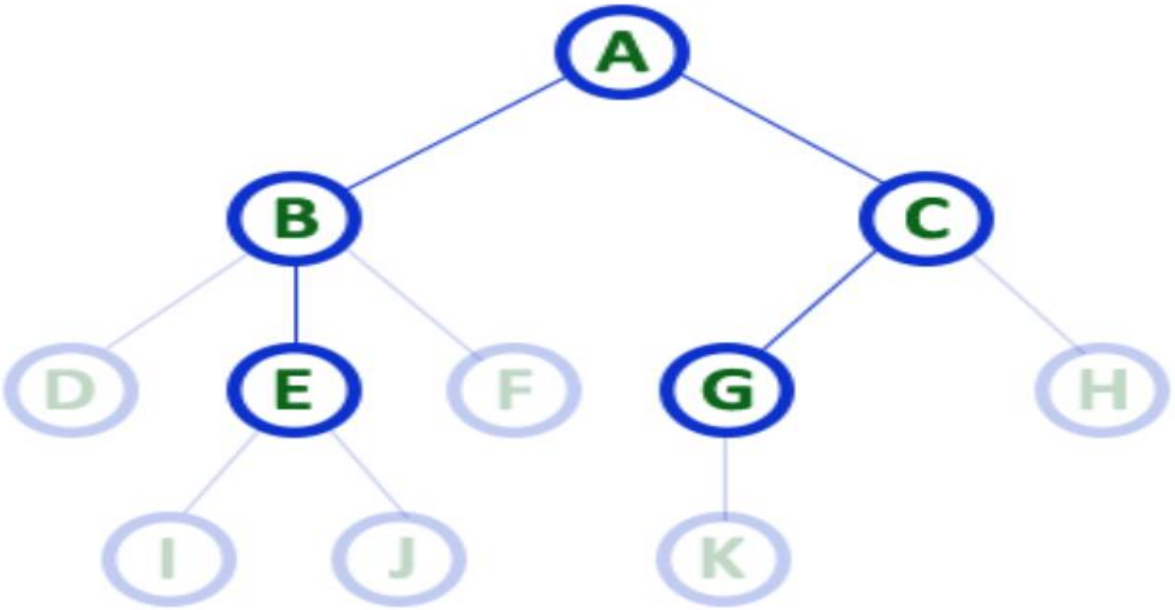
Formal Definition of Tree

- Defined as the finite set of one or more nodes such that there is a specially designated node called root and the remaining nodes are partitioned into n>= 0 different sets T_1, T_2, T_3……..Tn where T_1, T_2, T_3……..Tn called as Sub Trees.

Tree Terminologies

- **Root**
  - Root node is the origin of the tree data structure
  - In any tree, there must be only one root node.
- **Edge**
  - The connecting link between any two nodes is called as **EDGE**.
- **Parent**
  - Node which is a predecessor of any node is called as **PARENT NODE or** node which has child / children"

Parent Nodes

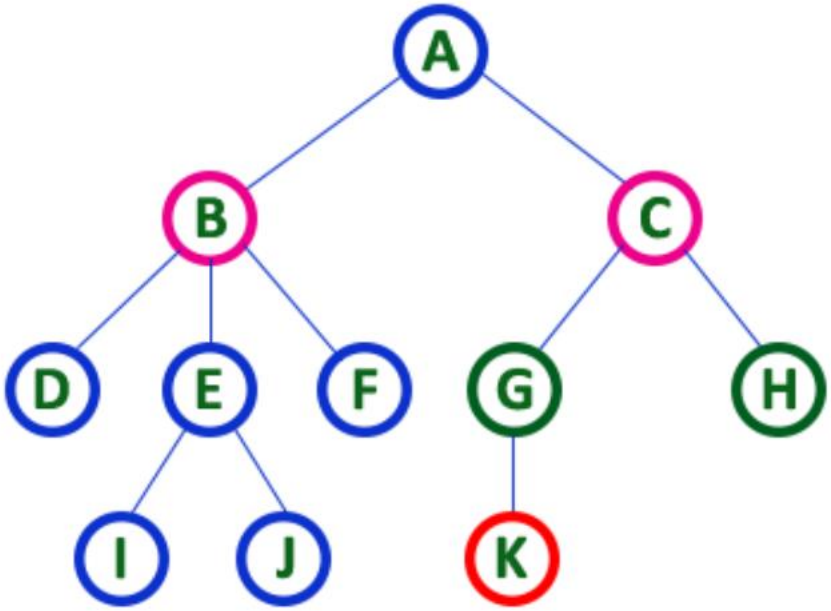


Child Nodes-node which is descendant of any node is called as **CHILD Node.**

In simple words, the node which has a link from its parent node is called as child node.

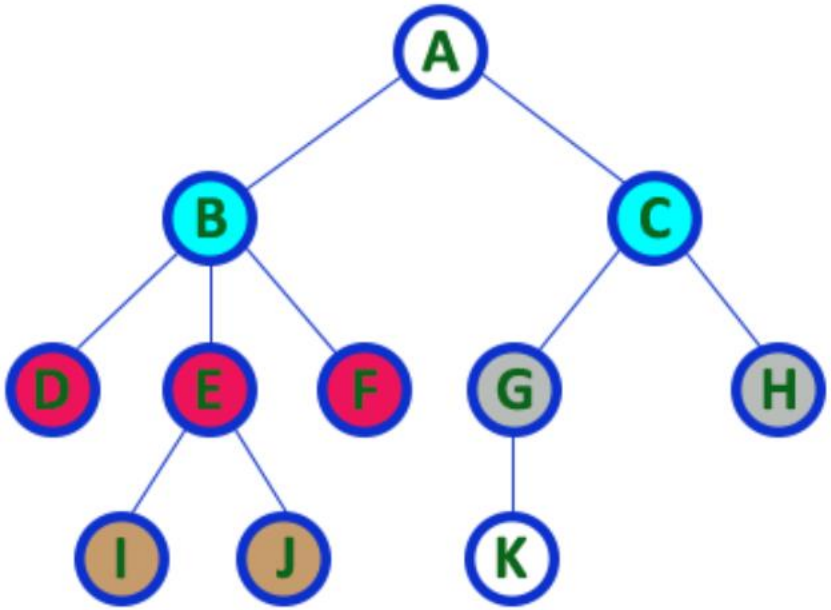
All the nodes except root are child nodes.

B and C are Children of A etc



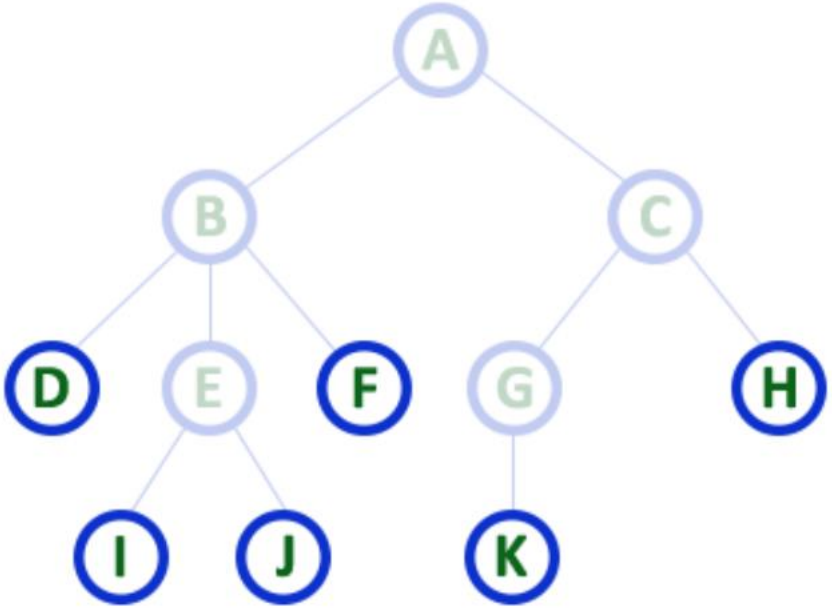
Siblings

- Nodes which belong to same Parent are called as **SIBLINGS**.
- B and C are Siblings of A etc…..



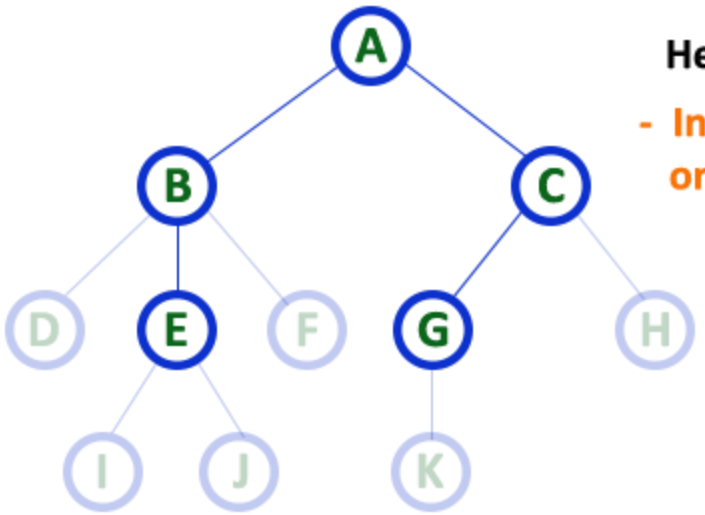
Leaf/ External Nodes/ Terminal nodes

- Node which does not have a child is called as **LEAF Node**



Internal Nodes/Non Terminal Nodes

- Node which has at least one child is called as **INTERNAL Node**.



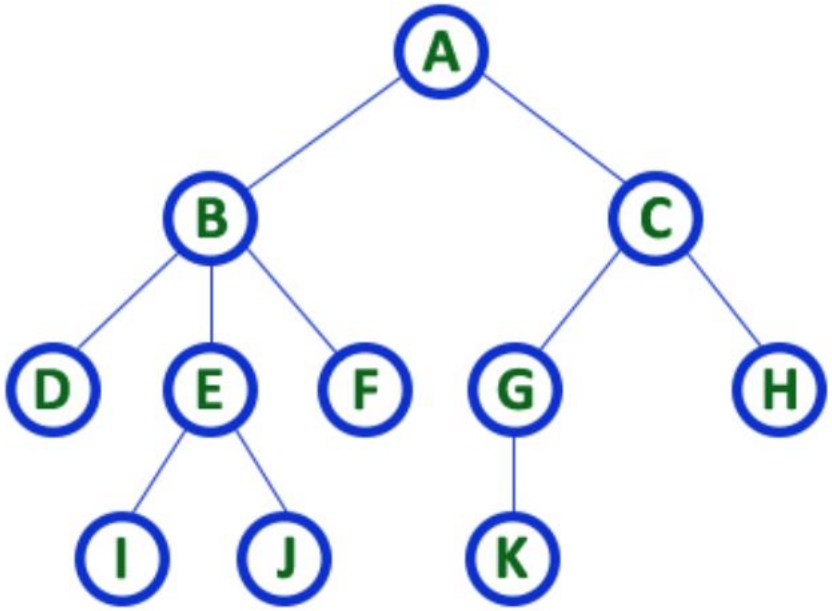
Here A, B, C, E & G are Internal nodes

- In any tree the node which has atleast one child is called 'Internal' node

- Every non-leaf node is called as 'Internal' node

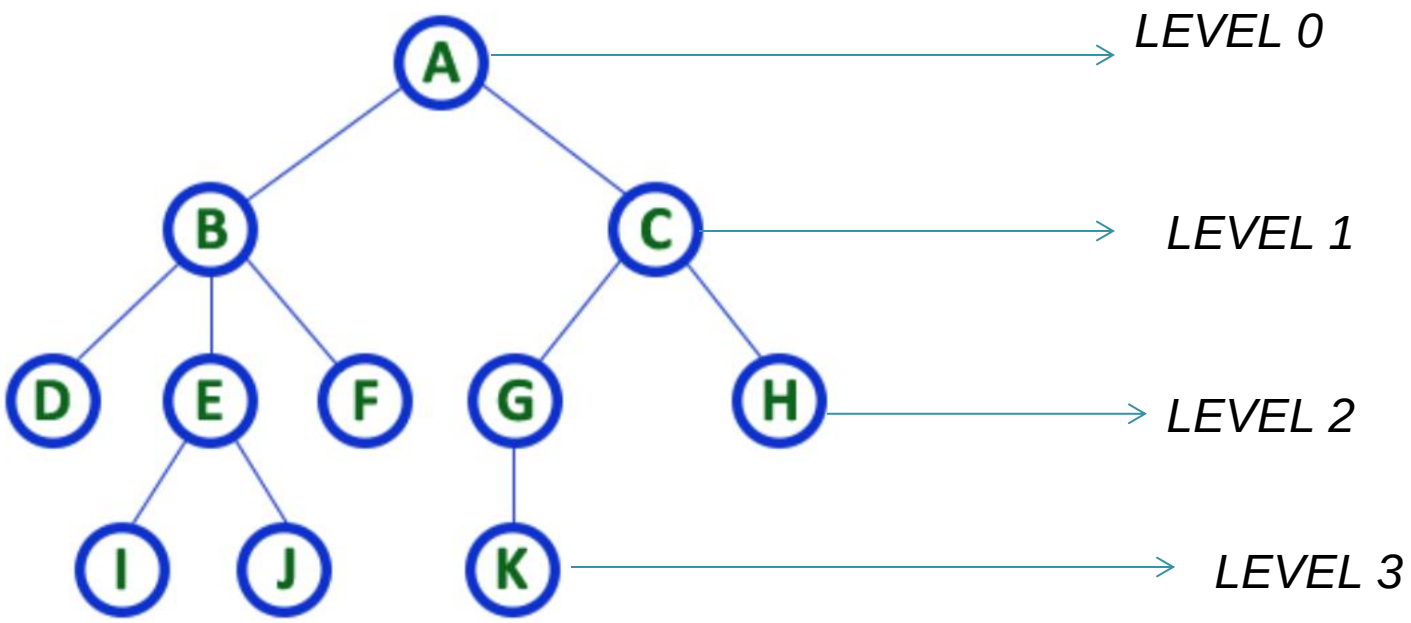
Degree

- Total number of children of a node is called as **DEGREE** of that Node.



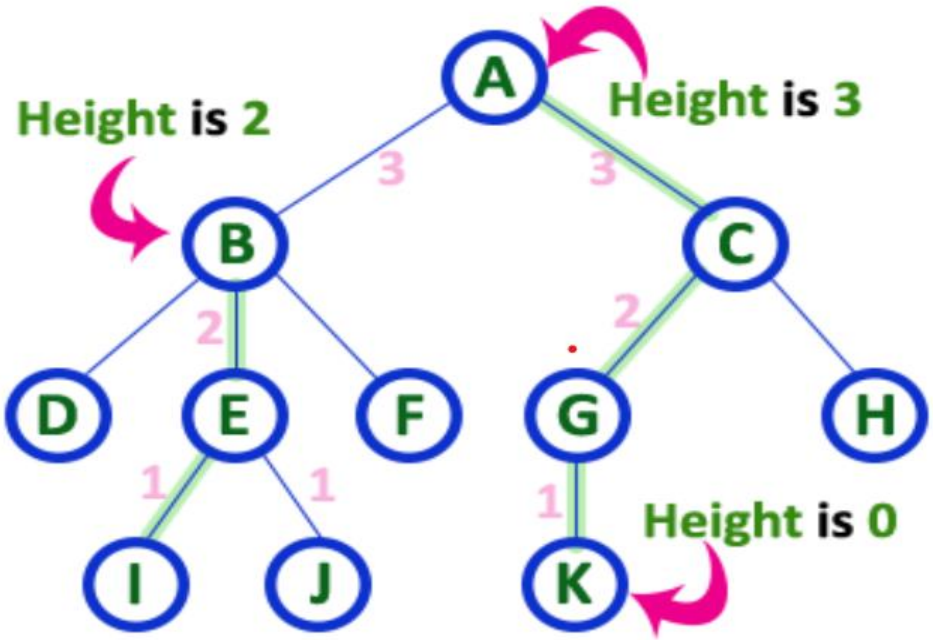
Here Degree of B is 3
Here Degree of A is 2

Level-- each step from top to bottom is called as a Level and the Level count starts with '0' and incremented by one at each level

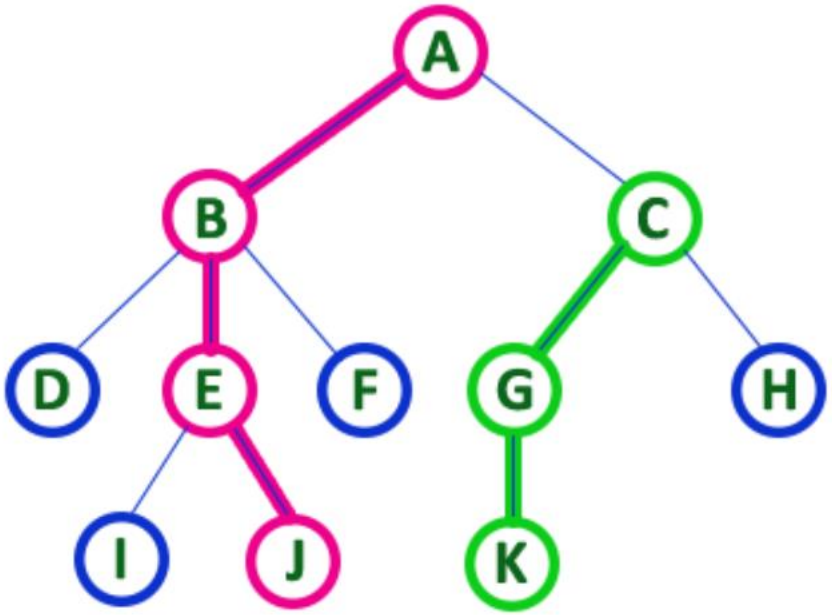


Height

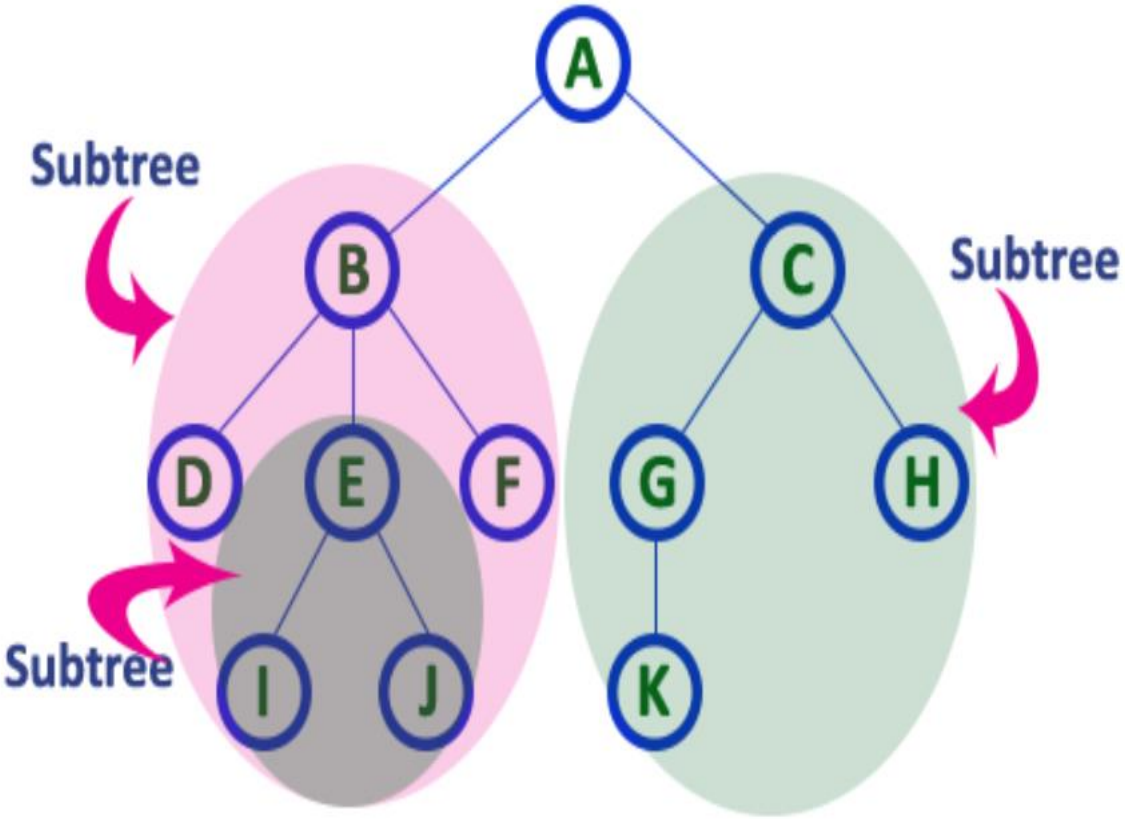
- Total number of edges from leaf node to a particular node in the longest path



Path - sequence of Nodes and Edges from one node to another node is called as **PATH** between that two Nodes



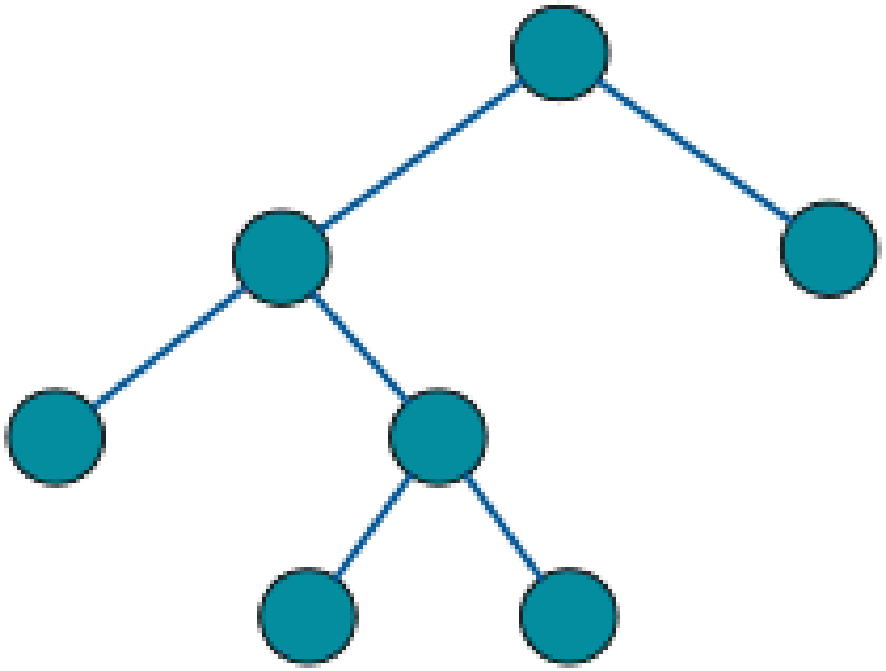
Sub Tree - Each child from a node forms a subtree recursively



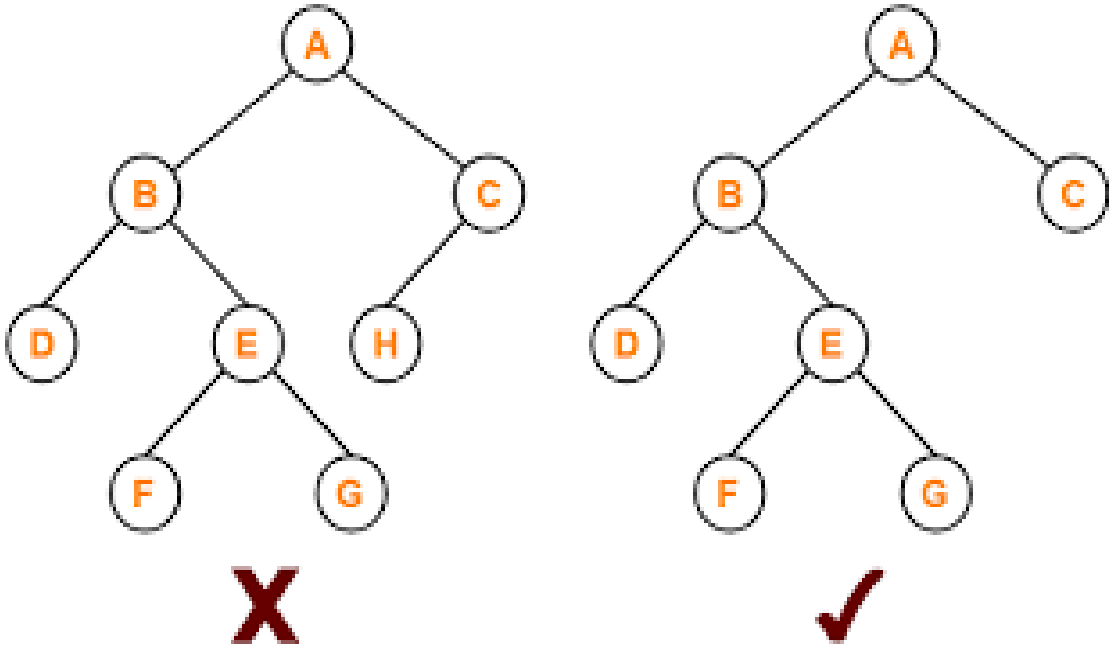
Binary Tree

- In a normal tree, every node can have any number of children.
- A binary tree is a special type of tree data structure in which every node can have a **maximum of 2 children.**
- Left child and the other one is Right child
- Every node can have either 0 children or 1 child or 2 children but not more than 2 children

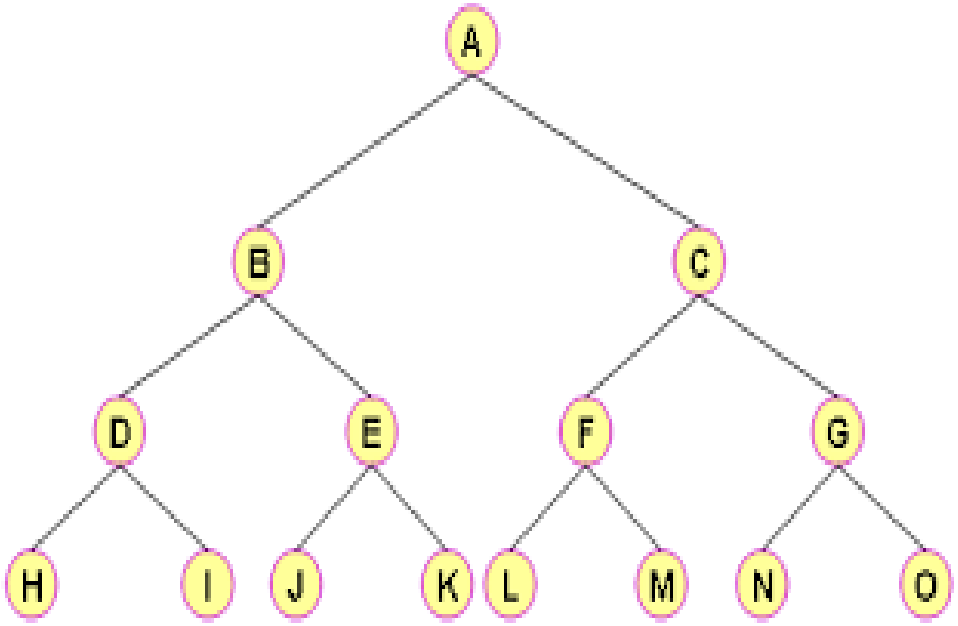
Binary Tree - Example



Strictly Binary Tree/Full binary tree/Proper Binary Tree/2-tree



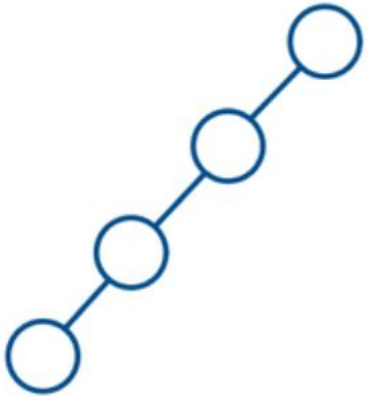
Complete Binary Tree/Perfect Binary Tree



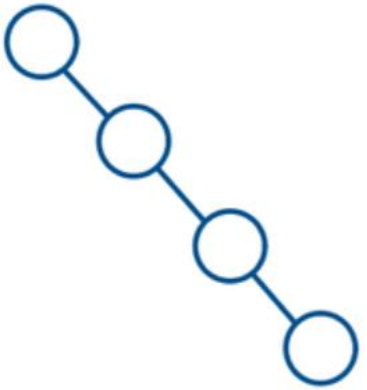
Skewed Binary Tree

- A skewed binary tree is a type of binary tree in which all the nodes have only either one child or no child.
  - **Left Skewed Binary Tree**
  - **Right Skewed Binary Tree**

Left Skewed



Right Skewed



Binary Tree Representation

- **Array Representation**
- **Linked List Representation**